VISION SYSTEM • STUDY NOTES

Indy7 Smart Cell 비전 빈피킹 시스템

YOLOv11-OBB · Intel RealSense D435 · Hand-Eye Calibration
합성 데이터 파이프라인부터 좌표 변환까지 한눈에

YOLOv11-OBB | RealSense D435 | Indy7 (IndyDCP3) | ArUco Calibration
Intel Arc A770 / IPEX | GroundingDINO | SAM | GrabCut

정성재 · Busan

학습용 정리노트 v1.0

목차

01 프로젝트 개요와 빈피킹이란? 3

02 전체 시스템 아키텍처 4

03 하드웨어 구성: Indy7 + RealSense D435 6

04 합성 데이터 파이프라인 8

05 YOLO 학습 (Intel Arc A770 + IPEX/XPU) 11

06 YOLOv11-OBB: 회전 바운딩 박스의 원리 13

07 Hand-Eye Calibration (ArUco 기반) 15

08 좌표 변환: 픽셀에서 로봇 좌표까지 17

09 빈피킹 실행 워크플로우 19

10 핵심 개념 학습 노트 (FAQ) 21

01 프로젝트 개요와 빈피킹이란?

한 줄 요약 카메라로 무작위로 놓인 부품을 인식하고, 로봇이 정확한 위치·각도로 집어올리는 시스템.

왜 만들었나?

산업 현장에서 부품을 컨베이어나 박스에서 집어 다음 공정으로 옮기는 작업은 매우 흔하지만, 부품이 **무작위로 흩어져 있고**, **회전된 상태**이며, **겹쳐 있을 수도 있는** 환경에서는 단순한 좌표 지시로는 풀 수 없다. 이 프로젝트는 협동로봇 **Indy7**과 깊이 카메라 **Intel RealSense D435**를 결합하여, AI 비전으로 부품의 위치·방향을 추정하고 로봇이 자동으로 집어들도록 만든 시스템이다.

"빈피킹(Bin Picking)"이란?

영어 그대로 "박스(bin)에서 집기(picking)" — 무작위로 담긴 부품을 로봇이 비전으로 판단하여 집어내는 작업. 난이도는 다음 순서로 올라간다:

1

2D Bin Picking

평면 위 부품의 (x, y, 회전각)만 추정. 본 프로젝트 범위.

2

2.5D Bin Picking

여기에 깊이(z) 정보를 추가. 부품 높이가 다를 때 사용.

3

3D Bin Picking

완전 무작위 자세(6-DoF). 가장 어려움, 포인트클라우드 기반.

핵심 도전 과제

① 부품의 **회전각**까지 정확히 추정해야 한다 (그래야 그리퍼가 올바르게 집을 수 있음)
② 카메라가 보는 픽셀 좌표를 **로봇이 이해하는 3D 좌표**로 변환해야 한다
③ 학습용 데이터가 부족할 때 **합성 데이터**로 모델을 학습시켜야 한다

이 문서에서 다루는 내용

1) 비전 모델 학습용 합성 데이터를 어떻게 만드는가 (GrabCut → SAM → GroundingDINO 발전사), 2) YOLOv11-OBB를 Intel Arc GPU에서 어떻게 학습시키는가, 3) ArUco 마커로 Hand-Eye Calibration을 어떻게 수행하는가, 4) 픽셀 좌표를 로봇 좌표로 변환하는 수학, 5) 실제 빈피킹 동작 흐름을 차례로 정리한다.

02 전체 시스템 아키텍처

먼저 큰 그림부터 보자. 시스템은 크게 **3계층**으로 구성된다.

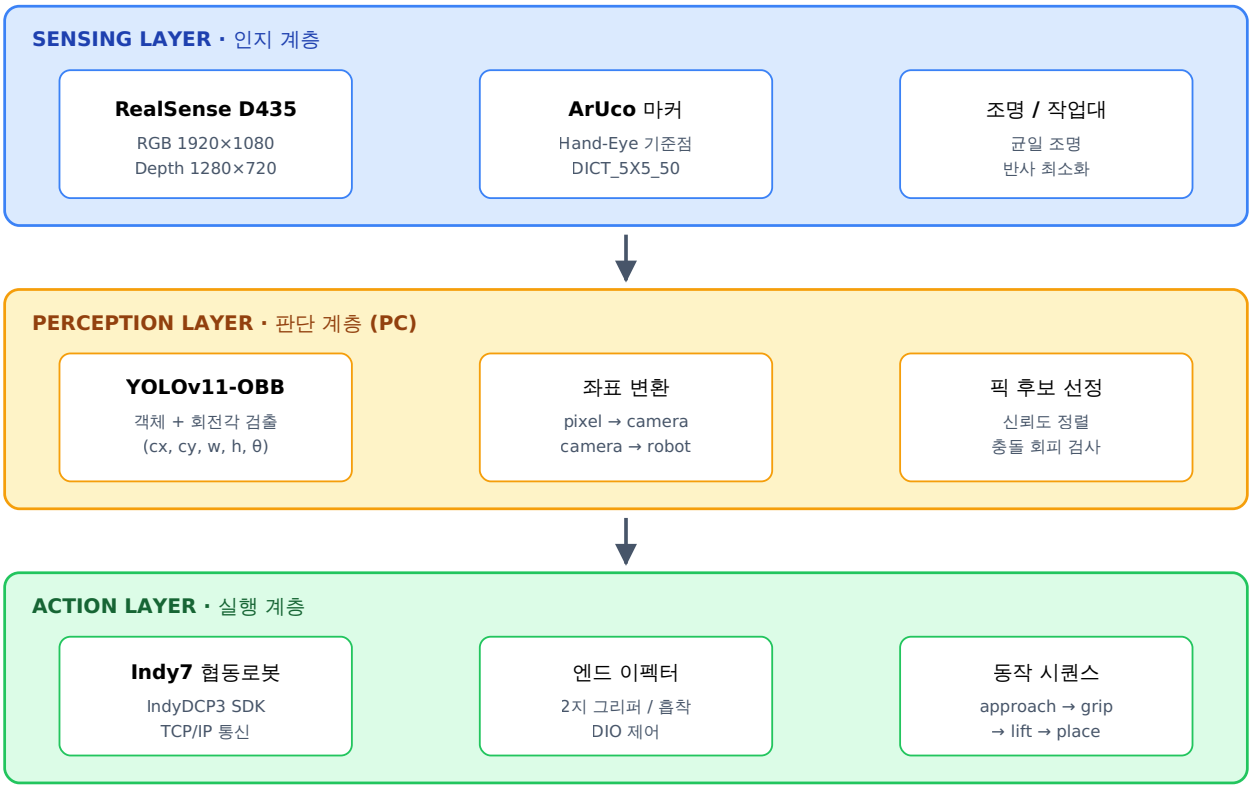


그림 2-1. 3계층 아키텍처 — Sensing(인지) · Perception(판단) · Action(실행)

왜 PC 오케스트레이션 구조인가?

Indy7 컨트롤러 안에서 비전을 직접 돌릴 수 없고, RealSense는 USB3 호스트가 필요하다. 따라서 **중앙 PC**가 컨트롤타워 역할을 한다:

- `pyrealsense2` — RealSense 카메라 스트리밍 (RGB + Depth)
- `ultralytics` — YOLOv11-OBB 모델 추론
- `IndyDCP3` — Indy7과 TCP/IP로 명령·상태 송수신
- `opencv` — ArUco 검출, 영상 처리

> **학습 포인트**
>
> "센싱 → 판단 → 실행"의 3단계 파이프라인은 **거의 모든 로보틱스 시스템의 기본 구조**다. 각 계층 간 인터페이스(데이터 형식·통신 방식)를 잘 정의해두면 부품 교체가 쉬워진다. 예: YOLO를 SAM2로 바꾸거나, Indy7을 다른 로봇으로 바꿔도 다른 계층은 그대로.

03 하드웨어 구성: Indy7 + RealSense D435

Intel RealSense D435 — 깊이 카메라

스테레오 기반 깊이 카메라. 좌우 IR 카메라의 시차(disparity)로 깊이를 계산하고, 별도의 IR 프로젝터로 패턴을 쏘아 텍스처가 없는 표면에서도 깊이를 얻는다.

항목	스펙	비전 빈피킹에서의 의미
RGB 해상도	1920×1080 @ 30fps	YOLO 입력 (보통 640×640으로 리사이즈)
Depth 해상도	1280×720 @ 30fps	z 값 추출, 픽킹 높이 계산
측정 거리	0.3m ~ 3m	작업 영역 30cm 이상 띄워야 함
FOV (D)	87° × 58°	워크스페이스 전체가 들어오도록 배치
인터페이스	USB 3.0	케이블 길이·노이즈 주의

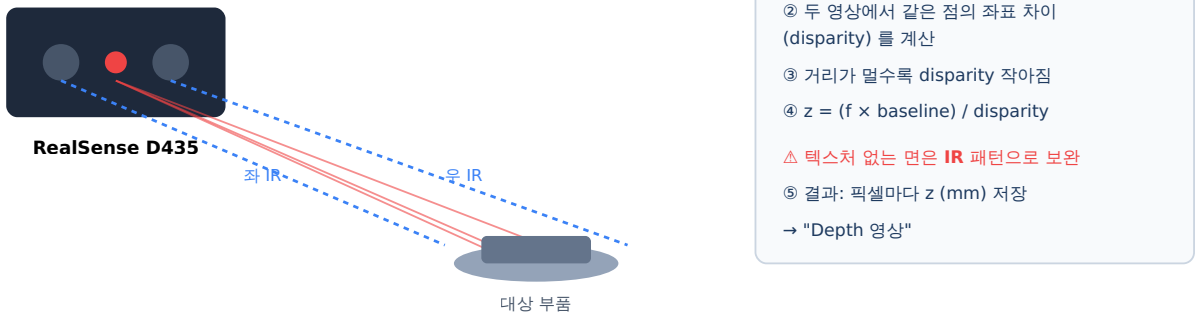


그림 3-1. RealSense D435의 스테레오 + IR 프로젝터 방식

카메라 장착 방식: Eye-to-Hand vs Eye-in-Hand

A

Eye-to-Hand (고정)

카메라가 작업대 위쪽에 **고정**되어 있고 로봇과 분리. 시야 안정적, 캘리브레이션 1회로 끝.

본 프로젝트가 채택한 방식.

B

Eye-in-Hand (장착)

로봇 손목에 카메라가 달려 같이 움직임. 좁은 공간 접근 가능하지만 매번 자세에 따라 변환 갱신.

Indy7 협동로봇

- 뉴로메카(Neuromeka)의 6자유도 협동로봇, 가반하중 7kg

- 제어: `IndyDCP3` 파이썬 SDK로 TCP/IP 명령
- 주요 API: `movej` (조인트), `movel` (직선), `get_control_state()`
- 충돌 감지 기능 내장 → 빈피킹에서 안전성 확보에 유용

```python
# Indy7 연결 기본 예시
from neuromeka import IndyDCP3

robot = IndyDCP3("192.168.0.5")
robot.connect()

# 작업 좌표계 기준 직선 이동: [x, y, z, rx, ry, rz] (mm, deg)
target = [350, 0, 200, 180, 0, 0]
robot.movel(target)
```

04 합성 데이터 파이프라인

왜 합성 데이터? 실제 부품 사진을 수천 장 찍고 손으로 라벨링하는 건 비용·시간이 막대하다. 부품 이미지 몇 장만으로 **수만 장의 학습 데이터**를 자동 생성하는 게 합성 데이터 파이프라인.

핵심 아이디어: Cut & Paste

부품을 **배경에서 깔끔히 잘라낸 뒤**(분할/마스킹), 다양한 배경 이미지 위에 무작위 위치·회전·크기·조명으로 합성하면 수많은 학습 이미지를 만들 수 있다. 핵심은 "잘 잘라내기" — 즉 **마스크 품질**이다.

세 가지 마스킹 기법의 발전사

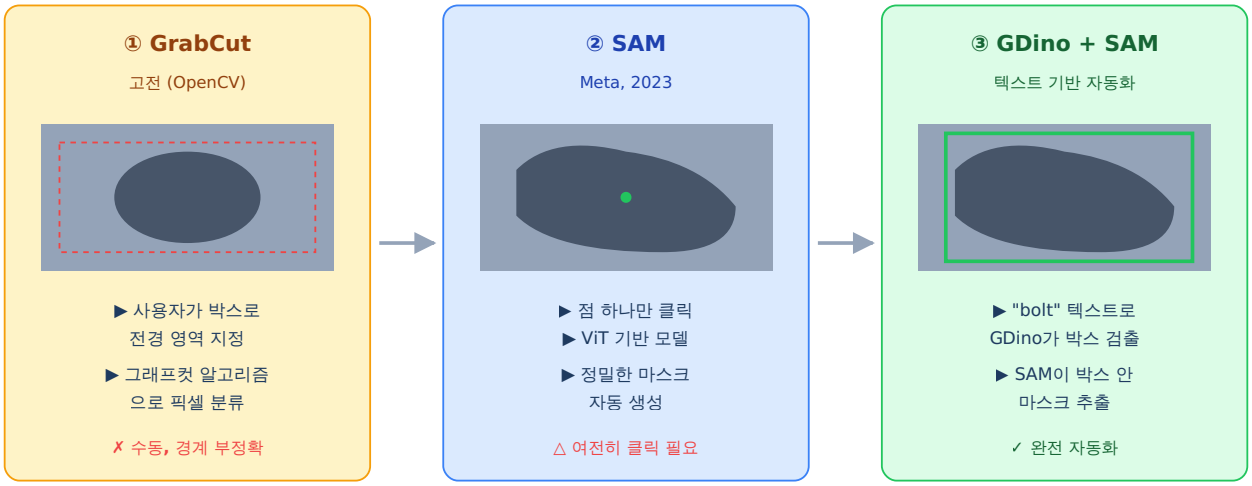


그림 4-1. 마스킹 기법 발전: GrabCut(수동) → SAM(반자동) → GroundingDINO+SAM(자동)

① GrabCut

OpenCV 내장 알고리즘. **그래프 컷(Graph Cut)**으로 전경/배경을 가우시안 혼합 모델로 학습하면서 분리한다. 사용자가 박스로 ROI를 지정하면 그 안에서 전경을 찾는 식. 단순한 부품엔 충분히 잘 동작하지만, 경계가 흐릿하거나 색이 비슷한 배경에선 품질이 떨어진다.

② SAM (Segment Anything Model)

Meta가 2023년에 공개한 **범용 분할 모델**. ViT 백본으로 학습되어 어떤 객체든 점 한 번 클릭이나 박스 입력으로 정확한 마스크를 생성한다. GrabCut 대비 경계가 훨씬 깨끗하고, 텍스처가 복잡한 부품에서도 잘 동작.

③ GroundingDINO + SAM (텍스트 기반 완전 자동)

GroundingDINO는 "**텍스트 입력으로 객체를 검출**"하는 모델(CLIP의 검출판이라 생각하면 된다). "bolt" 같은 텍스트를 주면 박스를 그려주고, 그 박스를 SAM에 넘기면 픽셀 단위 마스크가 나온다. → 사람 손이 전혀 필요 없는 완전 자동 라벨링 파이프라인.

```
# 합성 데이터 생성 코어 루프 (의사 코드)
for bg in backgrounds:
    for _ in range(samples_per_bg):
         img = bg.copy()
         labels = []
        for part in sampled_parts:
            # 1) 무작위 변환: 회전·스케일·밝기
              theta = np.random.uniform(0, 360)
              scale = np.random.uniform(0.7, 1.3)
              warped, mask = augment(part.img, part.mask, theta, scale)
            # 2) 마스크로 blend (포아송 블렌딩이 자연스러움)
              img = cv2.seamlessClone(warped, img, mask, center, cv2.MIXED_CLONE)
            # 3) OBB 라벨 기록
              labels.append(mask_to_obb(mask))   # (cx, cy, w, h, theta)
        save(img, labels)
```

실전 팁

1) Domain Randomization — 조명·배경·노이즈를 일부러 과하게 변주하면, 실제 환경에서도 잘 일반화된다.

2) Poisson Blending — 단순히 붙이지 말고 `cv2.seamlessClone` 의 MIXED_CLONE 모드로 자연스럽게 합성.

3) Sim-to-Real Gap — 합성 데이터만으로 학습한 모델은 실제에서 성능이 떨어지는 경향. 실제 이미지 10~20%를 섞어 fine-tune.

05 YOLO 학습 (Intel Arc A770 + IPEX/XPU)

왜 Intel Arc A770?

NVIDIA GPU 없이도 YOLO 학습이 가능하다는 걸 보여주는 좋은 사례. **Intel Arc A770**은 16GB VRAM을 갖춘 데스크탑 GPU로, NVIDIA보다 훨씬 저렴하면서도 딥러닝에 충분한 성능을 낸다. 다만 PyTorch에서 바로 쓸 수 없고 **IPEX (Intel Extension for PyTorch)**가 필요하다.

XPU란?

PyTorch는 디바이스를 `cpu`, `cuda`로 구분한다. Intel은 이를 확장해 `xpu`라는 디바이스 타입을 추가했다. 즉 `tensor.to("xpu")` 하면 Intel GPU에서 연산이 돌아간다.



그림 5-1. IPEX 스택 — PyTorch 코드가 Intel Arc GPU에서 돌아가는 경로

학습 코드 (Ultralytics YOLO)

```python
import torch
import intel_extension_for_pytorch as ipex   # IPEX import 필수
from ultralytics import YOLO

print(torch.xpu.is_available())   # True 확인

model = YOLO("yolo11n-obb.pt")    # OBB 사전학습 가중치

results = model.train(
    data="data/parts.yaml",       # 합성 데이터셋 yaml
    epochs=100,
    imgsz=640,
    batch=16,
    device="xpu",                 # ← 핵심: xpu 지정
    project="runs/obb",
    name="indy7_parts_v1",
)
```

학습 데이터 yaml 예시

```yaml
# data/parts.yaml
path: ./datasets/synth_parts
train: images/train
val:   images/val

names:
  0: bolt
  1: nut
  2: bracket
```

OBB 라벨 형식

일반 YOLO는 `class cx cy w h` (정규화된 값)인데, OBB는 **4개 꼭짓점 좌표**를 직접 적는다:

```
# class x1 y1 x2 y2 x3 y3 x4 y4   (모두 0~1 정규화)
0 0.231 0.412 0.298 0.405 0.305 0.472 0.238 0.479
```

자주 겪는 함정

① `import intel_extension_for_pytorch` 를 **YOLO보다 먼저** 해야 xpu가 활성화됨
② Arc GPU 드라이버 버전이 낮으면 OOM 또는 NaN loss 발생 — 최신 드라이버 필수
③ AMP(자동 혼합정밀도)는 IPEX에선 `torch.xpu.amp` 네임스페이스 사용

06 YOLOv11-OBB: 회전 바운딩 박스의 원리

왜 OBB(Oriented Bounding Box)가 필요한가?

일반 YOLO의 출력은 **축에 정렬된 사각형**(Axis-Aligned BB, AABB). 회전된 길쭉한 부품엔 큰 문제: 박스 안에 **객체가 차지하는 비율이 너무 작아져서** 그리퍼가 어디를 잡을지 모호하다. OBB는 박스가 회전할 수 있어 부품을 꼭 맞게 감싼다.

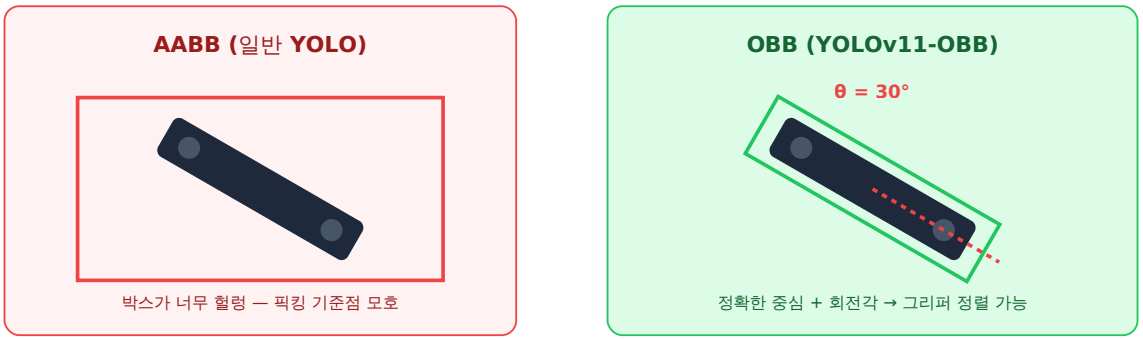


그림 6-1. AABB vs OBB. OBB는 객체의 회전각 θ까지 직접 출력

OBB의 5개 파라미터

$$OBB = (c_x, c_y, w, h, \theta)$$

중심점 좌표, 너비, 높이, 회전각

파라미터	의미	그리퍼와의 관계
c_x, c_y	박스 중심 픽셀	그리퍼 중심이 도달할 목표 위치
w, h	박스 너비/높이	그리퍼 폭 결정 (좁은 쪽이 잡는 방향)
θ	회전각 (라디안)	로봇 손목(엔드이펙터)을 같은 각도로 회전

회전각 표현의 모호함 (Boundary Discontinuity)

θ를 직접 회귀하면 문제가 생긴다. 0°와 180°는 길쭉한 부품에선 사실상 같은 자세인데, 손실 함수는 둘을 "180°만큼 다르다"고 본다. 이 불연속성 때문에 학습이 불안정해진다.

> **해결책: 각도 분류 (CSL, CIRCULAR SMOOTH LABEL)**
>
> θ를 **연속값으로 회귀**하지 말고 **180개 빈으로 분류**한다. 라벨도 1-hot이 아니라 주변에 가우시안으로 부드럽게 퍼뜨려서, 인접 각도는 비슷하게 학습되도록 한다. YOLOv11-OBB 내부에서 이런 트릭을 사용해 안정적인 학습을 달성한다.

추론 코드 예시

```python
from ultralytics import YOLO

model = YOLO("runs/obb/indy7_parts_v1/weights/best.pt")

results = model("frame.jpg", conf=0.6, device="xpu")

for r in results:
    obb = r.obb     # OBB 결과 객체
    for box, cls, conf in zip(obb.xywhr, obb.cls, obb.conf):
        cx, cy, w, h, theta = box.tolist()  # theta는 라디안
        print(f"class={int(cls)} conf={conf:.2f} 중심=({cx:.0f},{cy:.0f}) θ={np.degrees(theta):.1f}°")
```

07 Hand-Eye Calibration (ArUco 기반)

Hand-Eye Calibration이란? 카메라(Eye)와 로봇 손(Hand)의 **상대 위치·자세**를 알아내는 작업. 이게 없으면 카메라가 본 픽셀을 로봇이 갈 수 있는 좌표로 못 바꾼다.

왜 필요한가?

카메라는 자기 좌표계(camera frame)에서 부품의 3D 위치를 안다. 로봇은 자기 좌표계(base frame)에서 동작한다. 이 둘 사이의 변환행렬 $\mathbf{T}_{\text{camera}\to\text{base}}$를 모르면 둘은 영원히 다른 언어로 말한다.

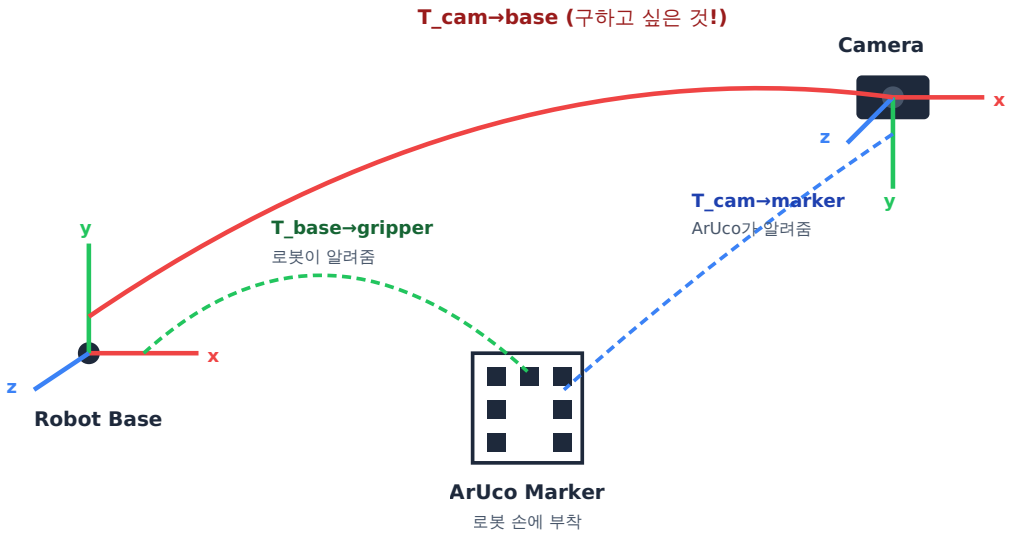


그림 7-1. 세 좌표계 — 로봇 베이스, 카메라, ArUco 마커

ArUco 마커란?

QR 코드처럼 보이는 흑백 패턴. 각 마커마다 고유 ID가 있고, 카메라가 보면 **마커의 4개 꼭짓점 픽셀 좌표**를 알 수 있다. 마커의 실제 크기(예: 50mm)를 알고 카메라 내부파라미터(intrinsics)가 있으면, OpenCV가 **마커의 3D 위치·자세를 카메라 좌표계에서 직접 계산**해준다 (PnP 문제).

캘리브레이션 절차 (Eye-to-Hand의 경우)

1. **로봇 손 끝에 ArUco 마커를 단단히 부착**
2. 로봇을 **다양한 자세(15~30가지)**로 움직이며 각 자세마다:
   - 로봇이 알려주는 `T_base→gripper` 저장
   - 카메라에서 검출한 `T_cam→marker` 저장
3. OpenCV의 `cv2.calibrateHandEye()` 호출 → `T_cam→base` 산출

$$AX = XB$$

A: 로봇 자세 간 변화, B: 마커 자세 간 변화, X: 구하려는 T_cam→base

핵심 코드

```python
import cv2
import numpy as np

# 각 캘리브레이션 자세마다 수집한 리스트들
R_gripper2base, t_gripper2base = [], []     # 로봇 forward kinematics 결과
R_target2cam,   t_target2cam   = [], []     # ArUco 검출 결과

# ... 여러 자세 수집 후 ...

R_cam2base, t_cam2base = cv2.calibrateHandEye(
    R_gripper2base, t_gripper2base,
    R_target2cam,   t_target2cam,
    method=cv2.CALIB_HAND_EYE_TSAI    # Tsai-Lenz 알고리즘
)

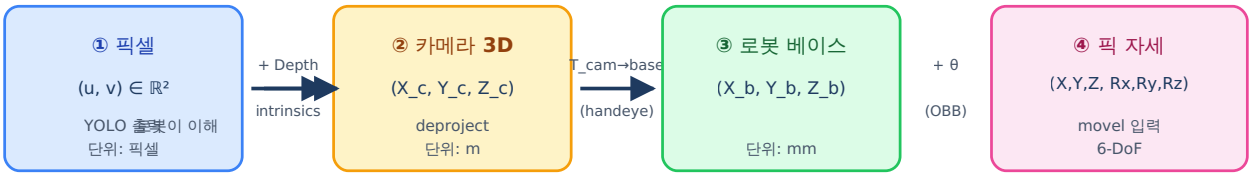
T_cam2base = np.eye(4)
T_cam2base[:3, :3] = R_cam2base
T_cam2base[:3,  3] = t_cam2base.flatten()
```

캘리브레이션 품질 팁

① 로봇 자세를 **3축 회전 모두 다양하게** 변주해야 정확도 ↑ (한 평면에서만 움직이면 부정확)
② 마커가 카메라 화면의 **구석·중앙** 골고루 분포해야 한다
③ 결과 검증: 마커를 알려진 위치에 놓고 카메라로 본 좌표가 실제와 ±2mm 이내인지 확인

08 좌표 변환: 픽셀에서 로봇 좌표까지

YOLO가 출력한 **픽셀 좌표 (u, v)**를 Indy7이 갈 수 있는 **로봇 좌표 (X, Y, Z)**로 바꾸는 전체 흐름. 이게 비전 빈피킹의 수학적 심장부다.



픽셀 → 카메라 **3D** → 로봇 좌표 → 그리퍼 자세

각 단계마다 필요한 정보가 다르다

그림 8-1. 픽셀→로봇 좌표 변환 4단계

① 픽셀 → 카메라 3D (Deprojection)

RealSense는 픽셀별 깊이값 Z_c를 안다. 카메라 내부파라미터(focal length f_x, f_y 와 주점 c_x, c_y)를 이용해 다음 식으로 변환:

$$X_c = (u - c_x) \times Z_c / f_x$$
$$Y_c = (v - c_y) \times Z_c / f_y$$
$$Z_c = depth(u, v)$$

```python
# pyrealsense2의 편리한 헬퍼 사용
import pyrealsense2 as rs

intrin = depth_frame.get_profile().as_video_stream_profile().get_intrinsics()
depth_m = depth_frame.get_distance(u, v)        # 미터 단위
point_cam = rs.rs2_deproject_pixel_to_point(intrin, [u, v], depth_m)
# point_cam = [Xc, Yc, Zc] in meters
```

② 카메라 3D → 로봇 베이스 (Hand-Eye 적용)

Chapter 7에서 구한 `T_cam→base` (4×4 동차변환행렬)을 곱한다:

$$[X_b, Y_b, Z_b, 1]^T = T_{cam \to base} \cdot [X_c, Y_c, Z_c, 1]^T$$

```
p_cam = np.array([Xc, Yc, Zc, 1.0])    # 동차좌표
p_base = T_cam2base @ p_cam                # 행렬 곱
X_b, Y_b, Z_b = p_base[:3] * 1000         # m → mm
```

③ 회전각 변환 (OBB의 θ → 그리퍼 회전)

OBB의 θ는 **카메라 영상 안에서의 회전각**이다. 카메라가 작업대 위쪽에서 거의 수직으로 내려다본다면 이 θ를 그대로 **로봇 손목 Z축 회전(R_z)**으로 사용 가능하다.

다만 카메라가 기울어져 있다면 `T_cam→base` 의 회전 부분으로 θ를 변환해야 한다.

④ 그리퍼 접근 자세 결정

픽 포인트 바로 위에서 접근하기 위해 z를 **50~100mm 띄운 approach point**를 먼저 만들고, 이후 천천히 내려간다.

```
# 최종 픽 자세 계산
pick_pose     = [X_b, Y_b, Z_b,          180, 0, np.degrees(theta)]
approach_pose = [X_b, Y_b, Z_b + 80,    180, 0, np.degrees(theta)]

robot.movel(approach_pose)
robot.movel(pick_pose)
robot.gripper_close()
robot.movel(approach_pose)          # 다시 위로
```

09 빈피킹 실행 워크플로우

이제 모든 부품을 합쳐서 실제 동작 루프를 그려보자.

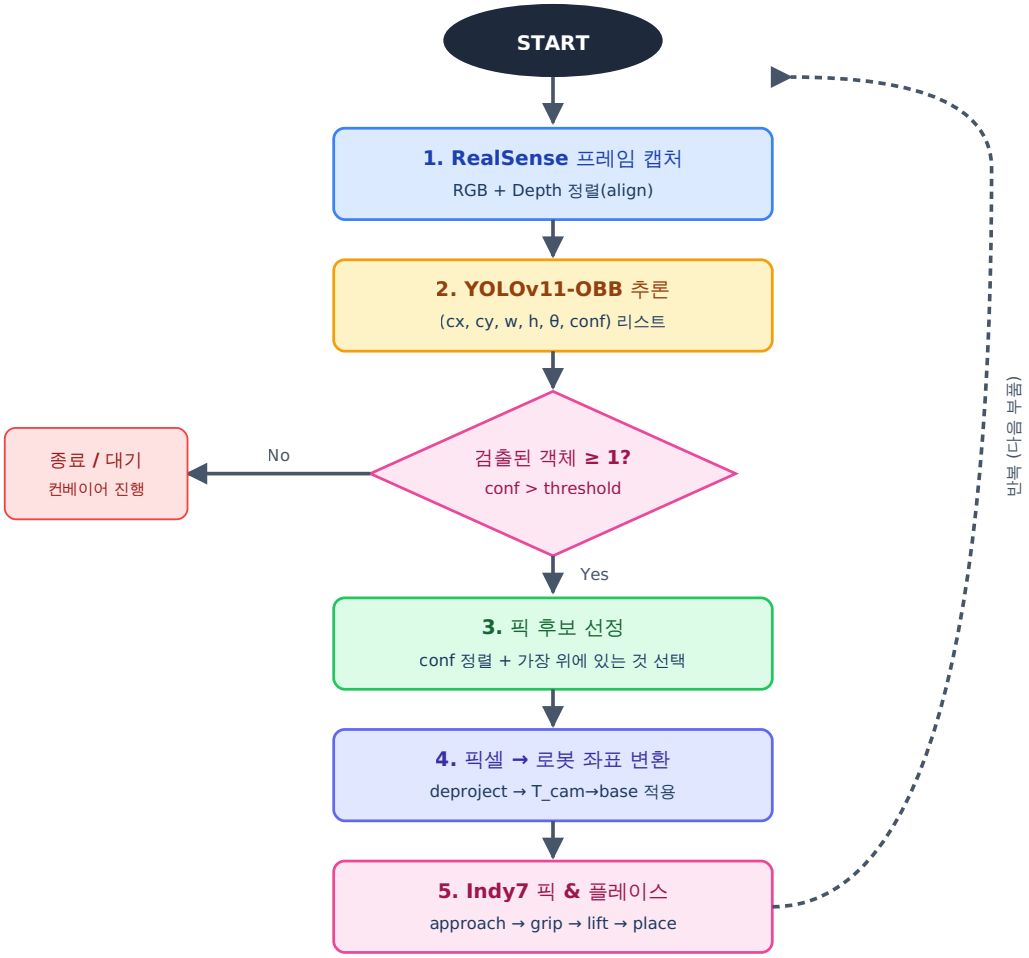


그림 9-1. 빈피킹 메인 루프 플로우차트

통합 코드 스켈레톤

```python
import intel_extension_for_pytorch as ipex
import pyrealsense2 as rs
import numpy as np
from ultralytics import YOLO
from neuromeka import IndyDCP3

# ──── 초기화 ────
model = YOLO("best.pt")
robot = IndyDCP3("192.168.0.5"); robot.connect()
pipe = rs.pipeline(); cfg = rs.config()
cfg.enable_stream(rs.stream.color, 1280, 720, rs.format.bgr8, 30)
cfg.enable_stream(rs.stream.depth, 1280, 720, rs.format.z16,  30)
pipe.start(cfg)
align = rs.align(rs.stream.color)
T_cam2base = np.load("handeye.npy")  # 사전 캘리브레이션 결과

while True:
    # 1) 캡처
    frames = align.process(pipe.wait_for_frames())
    color = np.asanyarray(frames.get_color_frame().get_data())
    depth = frames.get_depth_frame()
    intrin = depth.get_profile().as_video_stream_profile().get_intrinsics()

    # 2) YOLO-OBB
    res = model(color, conf=0.6, device="xpu")[0]
    if res.obb is None or len(res.obb) == 0:
        continue

    # 3) 최고 신뢰도 객체 선택
    best = res.obb.xywhr[res.obb.conf.argmax()]
    cx, cy, w, h, theta = best.tolist()

    # 4) 좌표 변환
    z_m = depth.get_distance(int(cx), int(cy))
    p_cam = rs.rs2_deproject_pixel_to_point(intrin, [cx, cy], z_m)
    p_base = T_cam2base @ np.array([*p_cam, 1.0])
    X, Y, Z = p_base[:3] * 1000          # mm

    # 5) 픽 실행
    pick = [X, Y, Z,    180, 0, np.degrees(theta)]
    appr = [X, Y, Z+80, 180, 0, np.degrees(theta)]
    robot.movel(appr); robot.movel(pick)
    robot.set_do(0, True)   # 그리퍼 close
    robot.movel(appr)
    robot.movel(place_pose)
    robot.set_do(0, False)
```

안전 체크 (실제 운영 시)

① 충돌 감지: Indy7의 내장 collision detection 켜기 (force threshold 30N 권장)
② 좌표 범위 검증: X, Y, Z가 작업 영역 밖이면 명령 전송 차단
③ depth 무효값: `get_distance` 가 0.0을 반환하면 (측정 실패) 그 후보는 건너뛰기
④ 비상정지 버튼은 항상 작업자 손이 닿는 위치에

10 핵심 개념 학습 노트 (FAQ)

Q1. YOLOv11과 YOLOv11-OBB는 뭐가 다른가?

모델 구조는 거의 같지만 **출력 헤드**가 다르다. 일반 YOLO는 (x, y, w, h, cls)를 출력하고, OBB 버전은 추가로 **회전각 θ**를 출력하는 헤드가 붙는다. 손실 함수도 회전을 고려한 IoU (예: ProbIoU, Rotated IoU)로 바뀐다.

Q2. ArUco 말고 ChArUco는 뭔가?

ArUco는 단일 마커, **ChArUco**는 체스보드 + ArUco를 결합한 보드. 카메라 **내부파라미터(intrinsics) 캘리브레이션**에 더 정확하다. Hand-eye 캘리브레이션(외부파라미터)에는 단일 ArUco도 충분하지만, 카메라 자체를 먼저 캘리브레이션할 때 ChArUco를 쓰면 좋다.

Q3. 카메라 내부파라미터(intrinsics)와 외부파라미터(extrinsics)의 차이?

Intrinsics (내부)

카메라 자체의 성질: focal length, principal point, 렌즈 왜곡 계수. **카메라가 바뀌지 않는 한 변하지 않는다.**

RealSense는 공장 캘리브레이션 값을 SDK로 제공.

Extrinsics (외부)

카메라가 **다른 좌표계(예: 로봇 베이스)와의 상대 위치·자세**. 카메라를 옮기면 다시 캘리브레이션 필요.

Hand-eye calibration이 구하는 것.

Q4. 합성 데이터만으로 충분한가? (Sim-to-Real Gap)

부족하다. 합성 이미지와 실제 이미지의 미세한 차이(조명, 노이즈, 색감) 때문에 합성만으로 학습한 모델은 실제 환경에서 성능이 떨어진다. 해결책:

1. **Domain Randomization**: 합성 시 일부러 조명/색감을 과도하게 변주
2. **Domain Adaptation**: 실제 이미지를 소량 추가하여 fine-tune
3. **스타일 변환**: CycleGAN 등으로 합성 → 실제 스타일로 변환

Q5. SAM 대신 SAM2를 쓰면 뭐가 좋나?

SAM2(2024)는 **비디오 분할**까지 지원하고 추론 속도가 6배 빠르다. 합성 데이터 파이프라인에선 단일 이미지로 충분하므로 SAM과 큰 차이 없지만, 연속된 프레임에서 같은 객체를 추적해야 할 땐 SAM2가 압도적으로 유리.

Q6. 왜 라디안과 도(degree)가 자꾸 헷갈리는가?

파이썬 생태계에선 **대부분 라디안이 기본**이다:

- YOLO OBB θ → **라디안**
- OpenCV의 회전 변환 → 라디안
- numpy/scipy 삼각함수 → 라디안
- 그러나 Indy7 `movel` 입력 → **도(degree)**

경계에서 변환을 잊으면 디버깅하기 어려운 버그가 생긴다. 항상 `np.degrees()` / `np.radians()` 로 명시적으로 변환.

Q7. 그리퍼가 부품을 가끔 놓치는데 원인은?

증상	원인 후보	해결
위치는 맞는데 살짝 빗나감	Hand-eye 정밀도 부족	캘리브레이션 자세 다양화, 자세 수 늘리기
회전각이 90° 틀어짐	OBB θ의 좌표계 정의 차이	θ에 -1 곱하거나 π/2 더해서 보정
z가 부정확	Depth 무효값 / 반사	주변 픽셀 메디안, 다른 후보 선택
접근 중 충돌	approach 높이 부족	50→100mm로 상향, 천천히 내려가기

Q8. 더 공부하려면?

- **3D Bin Picking으로 확장**: PVN3D, FFB6D 같은 6-DoF 자세 추정 모델
- **Point Cloud 직접 처리**: PointNet++, Open3D로 포인트클라우드 정합
- **강화학습 기반 그래스핑**: Dex-Net, GraspNet 데이터셋
- **VLA로 확장**: 이미 진행 중인 DOBOT VLA 프로젝트 — 자연어 명령 + 비전 + 액션

전체 요약 **합성 데이터 생성 → YOLO 학습 → 추론 → Hand-eye 변환 → 로봇 명령.** 이 5단계가 모든 비전 빈피킹의 공통 구조. 각 단계의 변형(SAM2, 3D 자세, VLA 등)이 곧 다음 프로젝트의 방향이 된다.